VERSION
14.0

AMS GRA ARCHITECTURE VOLUMES

Multi-Function Aperture (MFA) MPU

AFLCMC/WA

April 24, 2026

U.S. GOVERNMENT NOTICES: Furnished to the U.S. Government with UNLIMITED RIGHTS as defined in DFARS 252.227-7013, this takes precedence over any restrictive marking herein.

Table of Contents

1 Overview **1**
1.1 Purpose . . . 1
1.2 Scope . . . 1
1.3 Authoritative References . . . 1
1.4 Informative References . . . 2

2 Multi-Function Aperture **3**
2.1 MFA Compute . . . 4
2.1.1 MFA Compute Design Patterns . . . 4
2.1.2 MFA Function Development . . . 7
2.2 Resource Management . . . 8
2.3 Key Features . . . 14
2.4 OEM Functions and Local Functions . . . 17
2.5 Aperture Virtualization . . . 18
2.5.1 Virtual Aperture . . . 18
2.5.2 VA Tasking . . . 19
2.5.3 VA and Local Function Association . . . 19
2.5.4 VA Development Tool . . . 19
2.5.5 VA Database . . . 20
2.5.6 MFA Local Function Software Load Definition . . . 20
2.5.7 Inhibit Virtual Aperture (VA) . . . 21
2.6 Interface Diagram . . . 21
2.7 Interfaces . . . 23
2.7.1 MFA Encapsulation Layer (MEL) . . . 26
2.8 Resource Management Utilization of RF Reports . . . 30

3 Compliance **31**
3.1 MFA Requirements . . . 31
3.2 Development Kit (DK) Requirements . . . 37

A Acronyms **43**

B Revision History **46**

List of Figures

2.1.1 This Figure Depicts Three Different Patterns that Represent Methods for Physically Implementing an MFA within Agile Mission Suite (AMS) Government Reference Architecture (GRA). 5
2.2.1 Resource Management Layers 8
2.2.2 Resource Management for MFA Types 1, 2, and 5 10
2.2.3 Resource Management for MFA Type 3 11
2.2.4 Resource Management for MFA Type 4 13
2.6.1 MFA Interface Diagram 22

List of Tables

1.3.1 Authoritative References 2
1.4.1 Informative References 2

2.0.1 MFA Types 4
2.3.1 MFA Key Features 16
2.7.1 MFA Interface Table 26
2.7.2 MEL Key Features 27
2.7.3 IR MFA Fundamental Data Types 29
2.7.4 RF MFA Fundamental Data Types 30

3.1.1 MFA MPU Compliance Requirements 36
3.2.1 DK Compliance Requirements 42

B.0.1 Revision History Table 46

1.0 Overview

A Multi-Function Aperture (MFA) is a Level 3 Minimum Procurable Unit (MPU) in the Agile Mission Suite (AMS) Government Reference Architecture (GRA). MFAs provide the interface to the environment for the Mission System as well as pre-processing resources and, optionally, internal scheduling.

1.1 Purpose

The MFA MPU Volume introduces the MFA MPU, describes the key features, logical context, interfaces, compliance requirements, and enumerates the different Types of MFAs within AMS GRA. The key features provide design guidance describing functionality that is frequently provided by an MFA MPU. The interface diagram contains a graphical representation of the types of interfaces that AMS GRA defines for an MFA MPU, from the perspective of the MFA MPU. The MFA Interfaces represented in **Table 2.7.1** contains a textual representation of the interfaces and their descriptions. Compliance requirements describe what is required for the MFA MPU to achieve an AMS GRA compliance tier. The MFA volume also describes the relationships between an MFA and a Local Function.

1.2 Scope

The MFA Volume supports compliance assessment for AMS GRA adopters.

1.3 Authoritative References

Authoritative references are key documents that a procuring authority, Mission System Integrator (MSI), or MPU provider refers to when determining whether a given AMS GRA entity is compliant with AMS GRA business and architecture requirements. An AMS GRA entity is compliant when it provides the required prescriptive constructs (e.g. interfaces for an MPU). See **Table 1.3.1** for a list of authoritative references for the MFA MPU System Volume.

Standard	Title	Version/Date
AMS GRA Volumes	Compliance Assessment Approach Volume	14.0
AMS GRA Volumes	Mission System Volume	14.0
AMS GRA Volumes	Software Architecture Volume	14.0
OMS Standard	OMS Standard	Version identified by the MSI
AMS GRA Volumes	Time Distribution Volume	14.0
AMS GRA Volumes	IR MEL IDD	14.0
AMS GRA Volumes	RF MEL IDD	14.0

Standard	Title	Version/Date
AMS GRA Volumes	MFA Compliance Documentation Template	14.0
AMS GRA Models	AMS GRA Architecture Model	14.0

Table 1.3.1: Authoritative References Identify the Key Documents Necessary for Performing a Compliance Assessment of the MFA MPU.

1.4 Informative References

Informative references are supplemental documents that a procuring authority, MSI, or MPU provider refers to for more information about an AMS GRA entity or related entities. See **Table 1.4.1** for a list of informative references for the MFA MPU Volume.

Standard	Title	Version/Date
AMS GRA Volumes	Local Function MPU Volume	14.0

Table 1.4.1: Informative References Identify Documents a Procuring Authority, MSI, or MPU Provider Can Use for More Information About the MFA MPU and Related Entities.

2.0 Multi-Function Aperture

An MFA is a collection of analog and digital electronics, firmware and software, and general pre-processing resources that serve as the source of interaction between the free space environment and the Mission System. Unlike traditional subsystem architectures, these apertures are not dedicated to any particular function, but rather work with independently procurable Skills. An MFA may have physical components distributed across the Mission Platform. However, all MFAs have one or more MFA faces, which are sub-assemblies of an MFA with a single, contiguous Field of Regard (FoR). MFA faces are expected to vary in multiple ways, including physical size and shape, element count, capability, integration with Local Functions, and performance behavior. In addition, MFA faces may vary in performance due to damage or degradation after delivery. Data exchanges between the MFA and Services running in the Multi-Function Processor (MFP) layer are defined by the MFA Encapsulation Layer interface. Some MFAs manipulate their internal resources to provide software Skills with simultaneous, segmented, and/or interleaved access to the MFA. Allowing MFAs to manipulate their internal resources enables rapid capability insertion via software without modifying the MFA hardware or logic.

A Radio Frequency (RF) MFA consists of one or more arrays of antenna elements, RF Analog to Digital Converters (ADCs) and RF Digital to Analog Converters (DACs), associated Original Equipment Manufacturer (OEM) hardware, and OEM infrastructure required to operate the array. An RF MFA supports timeshared, segmented, and time-coincident operations by digitizing the RF energy.

An Infrared (IR)[1] MFA consists of optics, a Focal Plane Array (FPA) detector, associated firmware, some instantiation of general OEM infrastructure designed to operate the array, and may also consist of a window. An IR MFA may consist of multiple IR MFA faces mounted on a single gimbal, multiple IR MFA faces distributed across the platform, a combination of the two, or a single IR MFA face.

AMS GRA defines five Types of MFAs. Types 1, 2, and 3 represent RF MFAs while Types 4 and 5 represent IR MFAs. The definitions of each Type are in **Table 2.0.1**.

Type	Description
Type 1 MFA	Self-contained Programmable RF Subsystem
Type 2 MFA	RF with no Internal Scheduler and one or more faces
Type 3 MFA	RF with Internal Scheduling and one or more faces
Type 4 MFA	IR with Internal Scheduling and one or more faces

[1] The term Infrared (IR) or IR is used and accepted to explicitly include sensors operating in optical frequency ranges outside the physics definition of Infrared Spectrum.

Type	Description
Type 5 MFA	IR with no Internal Scheduler and one or more faces

Table 2.0.1: This Table Defines AMS GRA MFA Types 1, 2, 3, 4, and 5.

2.1 MFA Compute

MFA Compute is the set of processing resources, (i.e. GPPs, GPUs, and FPGAs), associated with an MFA. MFAs may host Local Functions. In order for the MFA to host a Local Function, the MFA may provide MFA Compute, implement the MFA Compute Interface and MFA Internal Data Interfaces, and provide an associated Development Kit (DK) for Local Function developers. The MFA Compute and MFA Internal Data Interfaces provide the Local Function with access to MFA time reference, MFA Data products, and MFA processing resources to enable lower-latency and/or higher-bandwidth access to MFA resources than is available to MFP Services. The MFA Compute is logically part of the MFA and can be physically implemented according to one of the three design patterns presented in **Section 2.1.1**.

2.1.1 MFA Compute Design Patterns

The MFA patterns in **Figure 2.1.1** represent methods of implementing an MFA within the AMS GRA. These patterns are independent from the hosting of OEM Functions and Local Functions, which can occur in any of the patterns.

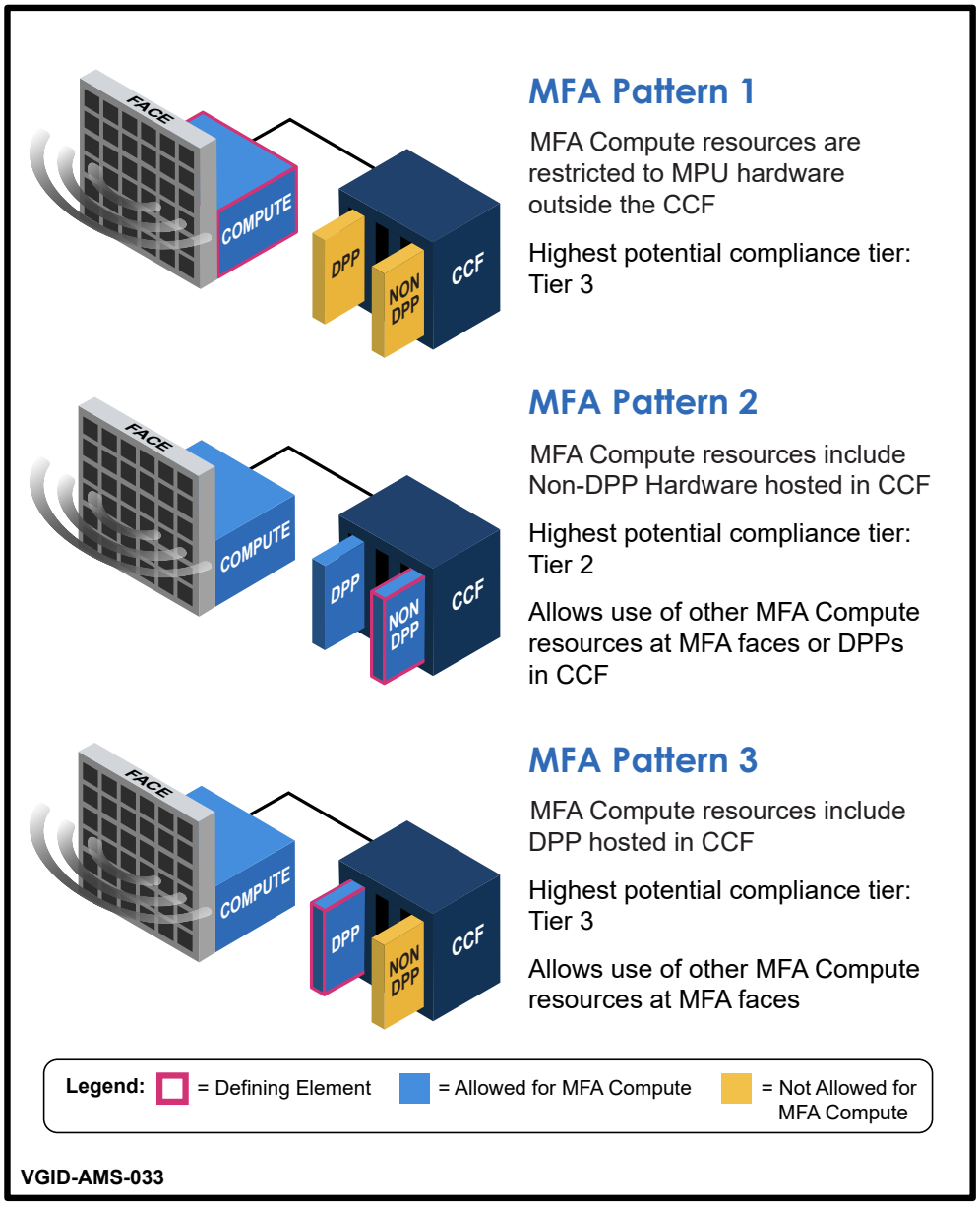


Figure 2.1.1: This Figure Depicts Three Different Patterns that Represent Methods for Physically Implementing an MFA within AMS GRA.

Pattern 1: MFA MPU is independent from CCF

Pattern 1 represents an MFA with no hardware dependencies on the Compute Cluster Farm (CCF). The MFA logical boundary contains MFA Compute residing with the MFA Face and/or in physically distributed subassemblies included with the MFA.

In Pattern 1, there are no other MPU dependencies and only MFA MPU compliance assessment occurs. According to the MFA MPU compliance requirements, the MFA compliance documentation captures the MPU design characteristics for a MSI to use.

Pattern 2: MFA MPU contains hardware installed in a Processor Chassis and Backplane (PCaB) of the CCF

Pattern 2 represents a physical instantiation of the MFA that contains a specialized hardware assembly that installs into a PCaB and is not a Digital Payload Processor (DPP). For Pattern 2, the MFA hardware is not limited to the AMS GRA definition of a DPP, so the hardware can be a purpose-built implementation. Pattern 2 depends on collaboration between the MFA provider and the MSI.

According to the MFA MPU compliance requirements, the MFA compliance documentation captures the MPU design characteristics for an MSI to use, to include the design characteristics associated with any specialized hardware located within the CCF to support integration of the Mission System.

MFA Pattern 2 Trust Interface Access

MFAs need to authenticate connections and protect sensitive data using a Trust Interface. Pattern 2 MFAs have non-DPP hardware within the PCaB. In this pattern, the MFA Face and the PCaB must use a Trust Interface to ensure sensitive data is protected within the MFA MPU boundary when the data crosses boundaries of other MPU.

Pattern 3: MFA MPU uses assignable DPP in the CCF

Pattern 3 represents an MFA implementation that uses a DPP in the CCF for MFA Compute. Over different periods of run time (e.g., the entire mission or a mission segment), the DPP may have the following assignments depending on resource availability:

- Exclusive assignment to an MFA
- Exclusive assignment to the MFP/Mission Processing Subsystem (MPS)
- Shared assignment between an MFA and the MFP/MPS

Pattern 3 depends on collaboration between the MFA provider, the MSI, and also potentially includes the DPP provider. Depending on the design approach, the following scenarios may apply to the identification and provision of the DPP:

- The MFA provider may also serve as the DPP MPU provider for the shared DPP.
- The MFA provider may identify the requirements for the DPP for another entity to provide, (e.g., MSI and DPP provider).
- The MSI may identify candidate DPPs in the CCF for an MFA provider to use.

The MFA MPU provider documents the use of a DPP as MFA Compute, including any dependencies that require exclusivity or allow sharing, during MFA compliance assessment. The MFA MPU provider also documents network and bandwidth support between the DPP used for MFA Compute and the MFA face or faces.

MFAs need to authenticate connections and protect sensitive data using a Trust Interface. Pattern 3 MFAs have MFA Compute resources within the MFA Face and on a DPP. In this pattern, MFA Compute resources in the MFA Face and in the DPP must each use a Trust Interface to ensure sensitive data is protected within the MFA MPU boundary when the data crosses boundaries of other MPUs.

2.1.2 MFA Function Development

OEM Functions and MFA Local Functions use MFA Compute, independent of an MFA's physical implementation pattern. As such, the MFA has the following responsibilities:

- Definition of the development environment
- Coordination of the OEM Functions and Local Function
- Management of the OEM Functions and Local Function

For MFA Design Pattern 3, the MFA provider uses relevant portions of the DK for DPPs, in addition to any other MFA-specific augmentations, to provide the DK for Local Functions. The MFA provider, in collaboration with the MSI, may elect to use Mission Agnostic Service Infrastructure (MASI) functionality to assist with deployment and management, but MASI use is not required.

2.2 Resource Management

Resource Management is the overall process of managing AMS GRA resources, as distributed across all elements. MFAs that contain internal scheduling contribute to overall Resource Management, and this section presents typical responsibilities for a MFA with internal scheduling. There are several categories of Sensor Resource Management, including the following:

- Compute Processing Resources
- Data Transfer and Memory
- RF/IR Resource Management (e.g. Spectral, COSITE, Inhibit, Blanking)
- Aperture Resource Allocations
- Power and Cooling

The primary differentiating factors between the types of MFAs with regards to Sensor Resource Management (SRM) are Aperture Virtualization, Internal Scheduling, and Measurement Scheduling/Synchronous Control which are described in further detail within this section.

This section describes the responsibilities of the MFA and MFP responsibilities with respect to the interactions between an MFA and MFP per **Figure 2.2.1**. For further details on MFP and MPS resource management, see the Resource Management Volume.

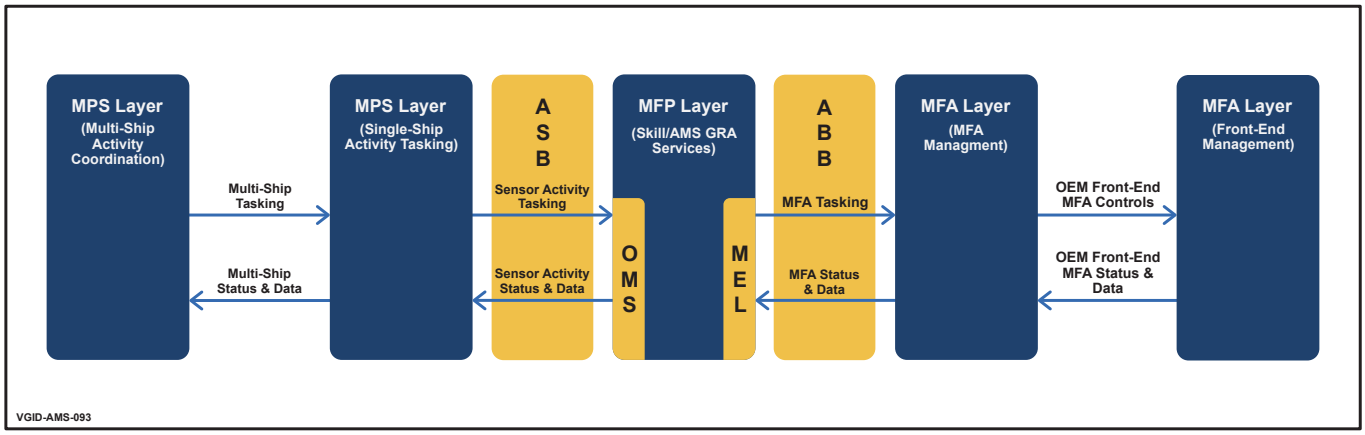


Figure 2.2.1: Resource Management Responsibilities Are Distributed Across the MFA, MFP, and MPS Layers. Note that this Figure Only Depicts MFA Layer Responsibilities for Type 3 MFAs.

The MFP layer (Skills/AMS GRA Services) is responsible for requesting Jobs/Tasks and providing priority information, as well as managing scheduler load. Similarly, the MFP layer is also responsible for providing a Window of Opportunity (WOO) or Time of Execution (TOE) for Jobs. The MFP layer communicates with the MFA over the Avionics Backplane Bus (ABB).

Type 1, 2 and 5 MFAs do not have internal scheduling, and rely on external resource management and deconfliction to have occurred before tasking. These MFAs do not have any resource management considerations, and operate as depicted in **Figure 2.2.2**.

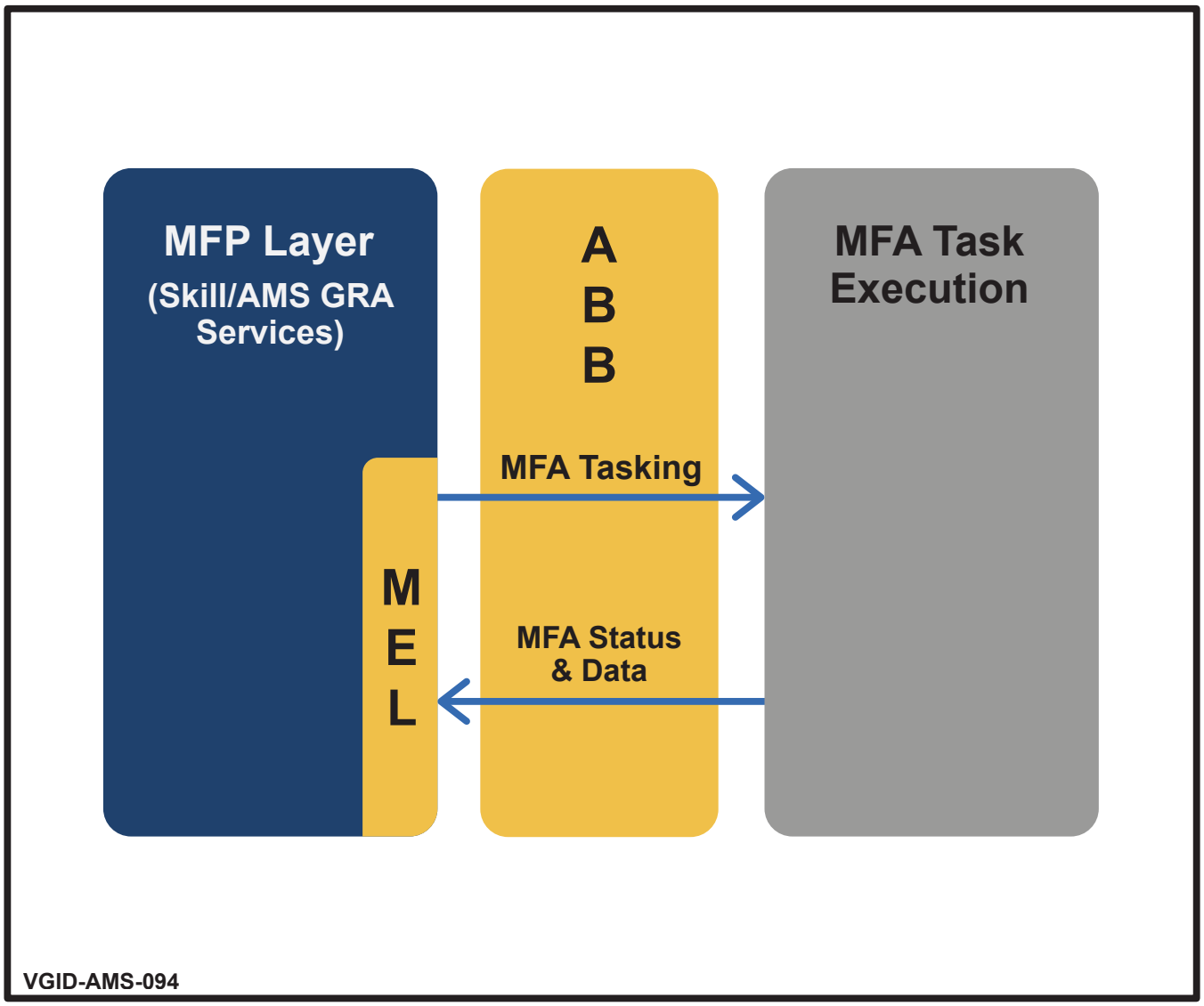


Figure 2.2.2: The MFP Layer Communicates via MEL over the ABB to Type 1, Type 2, and Type 5 MFAs. With MFA Tasking Information Provided by the MFP Layer, the MFA Layer Performs Task Execution and Transmits Status Information back to the MFP Layer via MEL.

Type 3 and 4 MFAs are defined by their internal scheduling, which adds SRM responsibility to the MFA.

Type 3 MFAs are responsible for internal scheduling related to job management. Responsibilities can include managing runtime job interleaving based on priority and resource conflict resolution (e.g. power or cooling limitations), or checking jobs for applicable/available RF bands before execution (e.g. rejecting a job from a Skill that incorrectly requested RF during Transmit Inhibit).

Type 3 MFAs will typically separate resource management into two layers. MFA management is typically responsible for approving or declining jobs, and scheduling execution In Accordance With (IAW) the requested WOO or TOE. Front-End Management is more hardware and firmware focused, and typically responsible for running job intervals and events within a job and maintaining hardware operability. Type 3 MFAs typically operate as depicted in **Figure 2.2.3**.

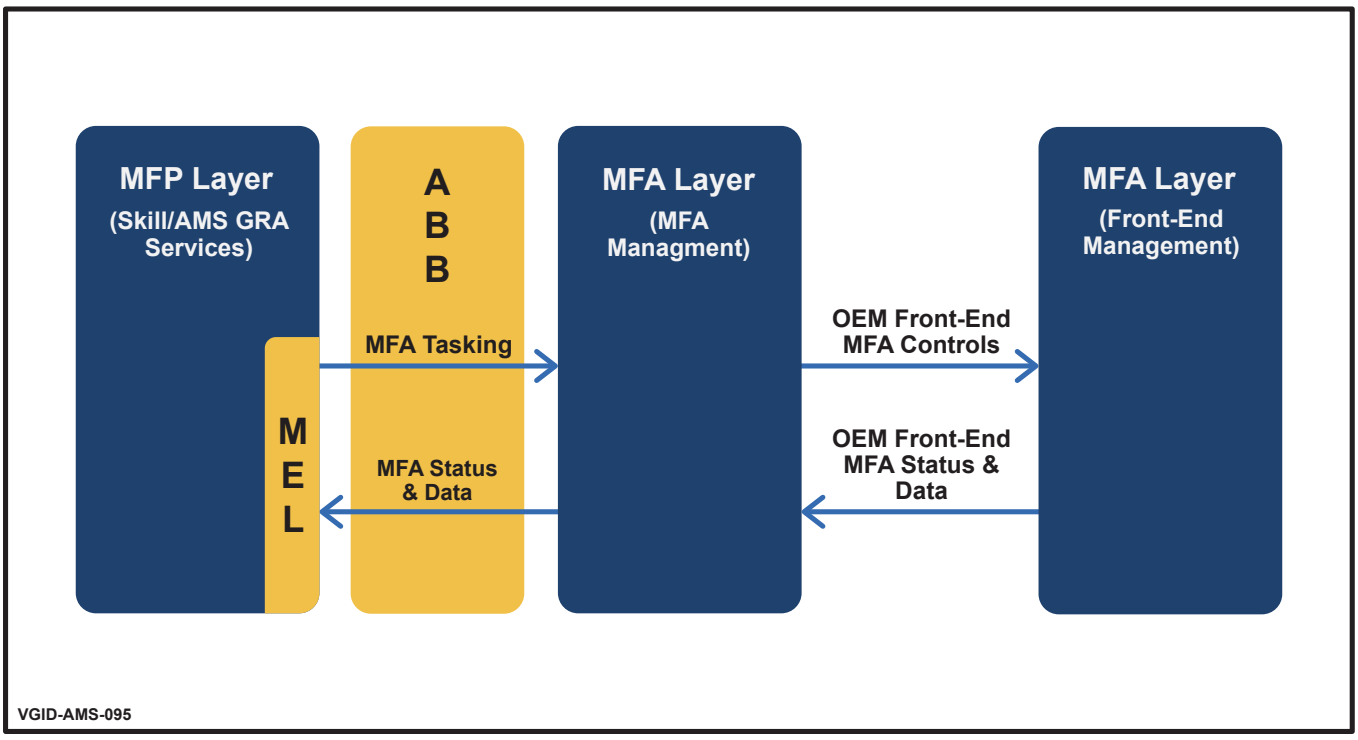


Figure 2.2.3: The MFP Layer Communicates via MEL over the ABB to Type 3 MFAs. One MFA Layer Is Used for MFA Management, while the Other Is Used for Front-End Management.

Type 4 IR MFAs are responsible for internal scheduling related to Task Management. These responsibilities vary across the three scheduling modes for IR MFAs: Task Scheduling, Scan Volume Scheduling, and Scan Bar Scheduling. Task Scheduling requires the least resource management and scheduling on the part of the MFA. The MFA is responsible for supporting tasks in priority order and executing the hardware functionality required to perform the command, but is not responsible for managing task scheduling.

Scan Volume and Scan Bar Scheduling require the MFA to deconflict resource consumption and schedule commands along a scan. With Scan Volume Scheduling the MFA has been provided a scan volume and the commands to perform, but is responsible for embedding commands within that scan volume to service all threats appropriately. With Scan Bar Scheduling, the MFA is responsible for all command scheduling, by using track priority and information supplied by the MFP to construct its own scan schedule. Type 4 MFAs typically operate as depicted in **Figure 2.2.4**.

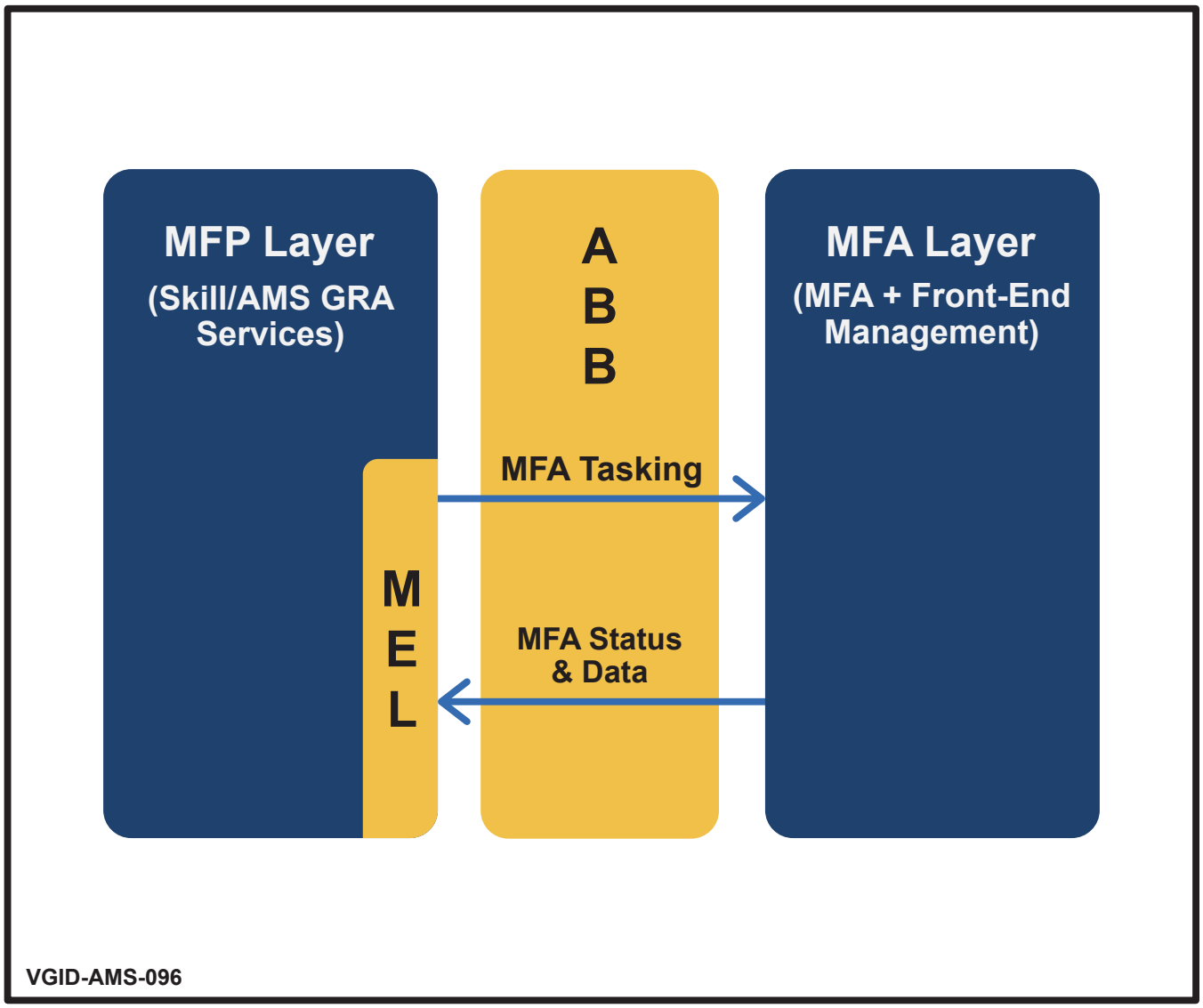


Figure 2.2.4: The MFP Layer Communicates via MEL over the ABB to Type 4 MFAs. The MFA Layer Performs both MFA and Front-End Management.

2.3 Key Features

Table 2.3.1 identifies key features of MFAs, the applicable MFA Types, and their benefits. This table is not prescriptive, but identifies a number of the features and attributes that AMS GRA interfaces are designed to enable. This table is included in the MFA documentation template so that MFA developers can summarize features supported by specific MFA instantiations.

Key Feature	Type 1	Type 2	Type 3	Type 4	Type 5	Benefit
Aperture Virtualization	No	Yes	Yes	No	No	Enables multiple Skills to use tailored configurations of MFAs as if they owned the aperture, subject to resource sharing constraints. Type 3 MFAs provide virtualization across array faces within a suite, and enable time-coincident sharing of physical resources across multiple activities and/or Skills.
Beamforming	Yes	Yes	Yes	No	No	Abstracts the internal array structure and application of the corrections required to achieve coherency into a data product for Capabilities that is consistent across RF MFA Types, bands, and implementations.
Built-In Test	Yes	Yes	Yes	Yes	Yes	Ensures the MFA is genuinely operational for the Capabilities; allows adaptation based on localized failures; this interface is provided by the MEL.
Calibration	Yes	Yes	Yes	Yes	Yes	Enables MFP hosted Capabilities to receive coherent data products from the MFA without exposing MFA internal details and forcing complex, redundant behavior on the Capabilities.
COSITE	Yes	Yes	Yes	No	No	All RF MFAs provide at least basic COSITE functionality. Type 3 MFAs feature internal scheduling combined with specific knowledge about element-to-element coupling and transmitted waveforms enables simultaneous transmit-and-receive.
Data Registration/Alignment/Correction Across Distributed Apertures	No	No	Yes	Yes	Yes	Ensures Skills receive consistently formed and corrected data; abstracts Skills for aperture installation locations enabling greater portability.
Data Registration/Alignment/Correction on Single Aperture	Yes	Yes	Yes	Yes	Yes	Ensures Skills receive consistently formed and corrected data.
Internal Scheduling	No	No	Yes	Yes	No	Type 3 and 4 MFAs provide internal scheduling that enables low-level timeline management and precise power management.
Measurement Scheduling /Synchronous Control	No	No	Yes	Yes	No	Optimizes utilization of limited scan time available to multiple Capabilities; aides in tracking targets across apertures.
Passive Detection Generation/Processing	Yes	Yes	Yes	Yes	Yes	Significantly reduces the data transport bandwidth required over the ABB to the Skills.

Key Feature	Type 1	Type 2	Type 3	Type 4	Type 5	Benefit
Power Down Declassification	Yes	Yes	Yes	Yes	Yes	Eases integration, maintenance, storage, and transportation of the MFA.
Power Management	Yes	Yes	Yes	No	No	For RF MFAs, power management provides a mechanism for precise control of power draw over time and array segments, particularly for active operation.
Secure Boot	Yes	Yes	Yes	Yes	Yes	Ensures the MFA boots into a secure, known state.
Stabilization	Yes	Yes	Yes	Yes	Yes	Ensures correct external orientation of data provided by the Skills.

Table 2.3.1: The MFA Key Features Table Describes the Specific Features Supported by an MFA Instantiation.

2.4 OEM Functions and Local Functions

An OEM Function within OEM Infrastructure includes software or firmware that is specific to an MFA implementation. This function, developed by the OEM, supports the safe operation of external interfaces, command and control, and internal device communications, status, built-in-test, and hardware-specific safeguards. This will allow the MFA to perform its role of providing shared access to the RF or IR aperture. OEM Functions support the basic Required and RequiredIf (driven by the MFA instantiation) capabilities described in the MFA Encapsulation Layer (MEL), including use cases of preventing commanded operations from causing damage or degraded performance of the hardware, configuring internal hardware for commanded operation, servicing external interfaces, and internal data management that is unique to the MFA design.

Local Functions are MPUs that implement logic within an MFA and are replaceable or upgradeable. Local Functions use internal MFA interfaces and hardware resources to perform a function, specific to a Skill. Local Functions primarily support two use cases, 1) enable lower latency response and autonomic capabilities not supported by latencies incurred across network transport, and 2) enable processing of data bandwidth that exceeds practical network limits. A Local Function may be statically or dynamically loaded into the MFA during run-time, based on system requirements and implementation constraints. Local Functions are developed using the MFA provided Local Function DK.

The following examples are provided to describe the intended functionality of OEM Functions and Local Functions. Functions marked with an asterisk (*) may be implemented as OEM Functions or Local Functions depending on the needs of an instantiation and may be available for upgrade.

Recommended OEM Functions

- Required MEL functionality
- Safe operation (e.g., blanking, interoperability)
- Hardware safeguards (e.g., transmitter state management)
- Validate simultaneous operations (Virtual Aperture (VA) compatibility)
- Internal hardware coordination (e.g., power, cooling)
- MFA status reporting
- Built-In Test (BIT)
- Hardware interfaces (e.g., RDMA over Converged Ethernet (RoCE), VME bus International Trade Association (VITA) 49.2)
- Internal configuration and data management
- Internal control and time distribution

- Controls scheduling and synchronization
- Reactive scheduler (e.g., dwell extensions)

Candidates for OEM Functions or Local Functions

- *Digital encoders/decoders (e.g., video codec, frame streaming)
- *Digital upconversion, downconversion, interpolation, decimation, filters, and mixer (In-Phase and Quadrature (IQ) is assumed baseband output, so general frequency/rate conversion is OEM)
- *Beamformers
- *Signal Generation (basic signal and waveform generation is required to be MEL compliant)
- *Pulse detection (for common skill uses)
- *Automatic Gain Control (for common skill uses)
- *Channelizer (for common skill uses)
- *Signal or video compression

Recommended Local Functions

- Low latency responses to RF environment, autonomic capabilities (decision logic)
- Low latency dynamic VA configuration switching
- Pulse measurement processing
- Enhanced/Specialty Signal generators
- Signal canceller algorithms
- Generally, single-skill specific logic

2.5 Aperture Virtualization

2.5.1 Virtual Aperture

Aperture Virtualization introduces the concept of a VA for RF MFAs to the architecture. VAs reflect the transformative nature of software-defined apertures that implement element-level or near-element-level digitization. These apertures provide an extraordinary level of configurability and adaptation that is highly parameter-driven. The set of MFA resources available extends beyond the number of elements in a subarray and includes (but is not limited to) ADC/DAC, transceiver, processing, Input/Output (I/O) bandwidth, and memory resources. However, the almost infinite number of resource combinations poses a significant complexity problem. The AMS GRA manages MFA complexity by using the concept of VAs. The VA construct within AMS GRA provides a Skill developer with the ability to define the subset of MFA resources, which enable the Skill to achieve performance objectives. This concept promotes the decoupling of Services from the specific design of an MFA. VAs are defined at design time and a Skill can use multiple VAs at runtime. An MFA may internally manage the instantiation and usage of VAs.

The VA represents a complete set of aperture resource types, quantities, Local Functions, and attributes required to support the needs of a specific Skill. A VA defines an abstracted set of MFA resources, which translate to hardware types, numbers, and relative locations. However, the MFA resources do not necessarily translate to physical components; instead, a VA instance could represent a specific set of physical components in the MFA that instantiate a defined VA. Most VAs will have multiple ways to physically instantiate a given VA on a digital

MFA. Following this paradigm, the VA presents to a Skill as a single taskable entity that may abstract the details of which physical hardware components implement the VA. Typically, software or firmware hosted within the logical boundaries of the MFA manages the VA.

2.5.2 VA Tasking

While one or more Skills may asynchronously task VAs, scheduling of each VA is potentially not independent. The internal MFA constraints may prevent use of different VAs at the same time (due to hardware restrictions and limitations). The MFA internal management functions may also modify which physical resources support a specific VA to improve optimization and simultaneous performance. This reconfiguration should continue to provide the Skill with the desired performance. A single VA may be instanced on a single MFA face, or may include elements on multiple MFA faces. In the case of a multi-faced VA, a Skill may select the MFA face to be used or may be abstracted from the specific installation details such that the transition from one MFA face to another occurs without Skill intervention.

2.5.3 VA and Local Function Association

Each VA may include the designation of one or more Local Function instantiations that augment the action of the VA with Skill-specific behaviors. The MFA is responsible for the lifecycle management of the Local Functions. The MFA deploys and initializes instantiations of each Local Function type associated with the VA.

MFAs that can host Local Functions may support deployment of Local Functions to a variety of locations within the MFA. These locations are dependent upon the MFA design and may vary significantly between different MFA designs. The information describing Local Function support and available deployment locations is documented by the MFA provider and communicated through the Local Function Development Kit. Additionally, the Local Function Development Kit describes what MFA Internal Data is available at each Local Function deployment location.

In RF MFAs, Local Functions are considered part of a VA definition and are loaded, deployed, and initialized when the associated VA is instantiated. Services can interact with a Local Function through the Local Function Command, included in the Job Interval class in the RF MEL Application Programming Interface (API). MFAs are responsible for writing the values set by the Service in this interface to the correct memory locations. MFAs are responsible for defining and documenting the mechanisms available to a Local Function for transmission of Local Function generated data out of the MFA to a defined Service endpoint. Additional information is provided in the RF MEL Interface Description Document (IDD)).

2.5.4 VA Development Tool

In addition to the VAs defined by the MFA, through the use of the VA Development Tool, the RF MFA suppliers may also define new VAs. The VA Development tool allows third-party developers to specify not only the public attributes of the new VA, but also the processing chains (e.g., Tx/Rx Element Groups, Endpoints, OEM Functions and Local Functions) within the MFA that implement the action of that VA. Since Local Functions must be associated with VAs via VA attributes, any MFA that allows third-party Local Function development must also provide a VA Development Tool.

2.5.5 VA Database

The Virtual Aperture Database (VADB) API, defined within the RF MEL, provides functions available to Skill developers at design time via an offline, MFA provider specific tool. This library is used by Skill providers to extract required, statically available VA information. The VADB includes the ability to load new VA Definitions. See the RF MEL IDD for more information about the VADB.

2.5.6 MFA Local Function Software Load Definition

The MFA is responsible for loading and deploying MFA OEM Functions and Local Functions to its MFA Compute using the Data Interface. Local Function image deployment is triggered when an AMS GRA Service request is made and access to the MFA VA is approved. Once the MEL has received and approved the request, the Local Function images are deployed. If the Local Function image is stored outside the MFA, then the requesting AMS GRA Service informs the MFA of the location of the Local Function image. The AMS GRA Service transmits the location via the MEL API content. Local Functions may be loaded at MFA startup or during operation as needed to support Skill execution.

The requesting AMS GRA Service is unlikely to know the precise location of Local Function images. The Mission System may assume the responsibility for storing and organizing Local Function images using MPUs such as Data Storage. In this case, the AMS GRA Service may use MASI or a configuration file to locate the Local Function image on the Mission System. For each Local Function associated with the needed VA, the requesting AMS GRA Service passes this information to the MFA through the MEL, as part of its request for the VA.

Additional detail on how the MEL interface supports Local Function download is available in the RF MEL IDD.

For RF MFAs, Skills can provide VA definition and Local Functions file storage and retrieval information to the MFA when requesting specific VAs using the RF MELs API. The MFA is responsible for the lifecycle management of VAs and Local Functions, including periodic status of these components to the requesting Skills.

2.5.7 Inhibit Virtual Aperture (VA)

AMS GRA also includes the concept of an Inhibit VA. The purpose of the Inhibit VA feature is to address COSITE concerns associated with RF compatibility across multiple MFAs. The Inhibit VA provides an interface for one or more Services to restrict transmissions on specific MFAs. The MSI is responsible for defining which Services can use Inhibit VAs.

2.6 Interface Diagram

Figure 2.6.1 depicts the MFA MPU and its external interfaces. The interface diagram only contains logical entities. Interface connections denoted as pass-through only represent situations where the sending MPU is not the original source and does not use or modify what is provided to the receiving MPU.

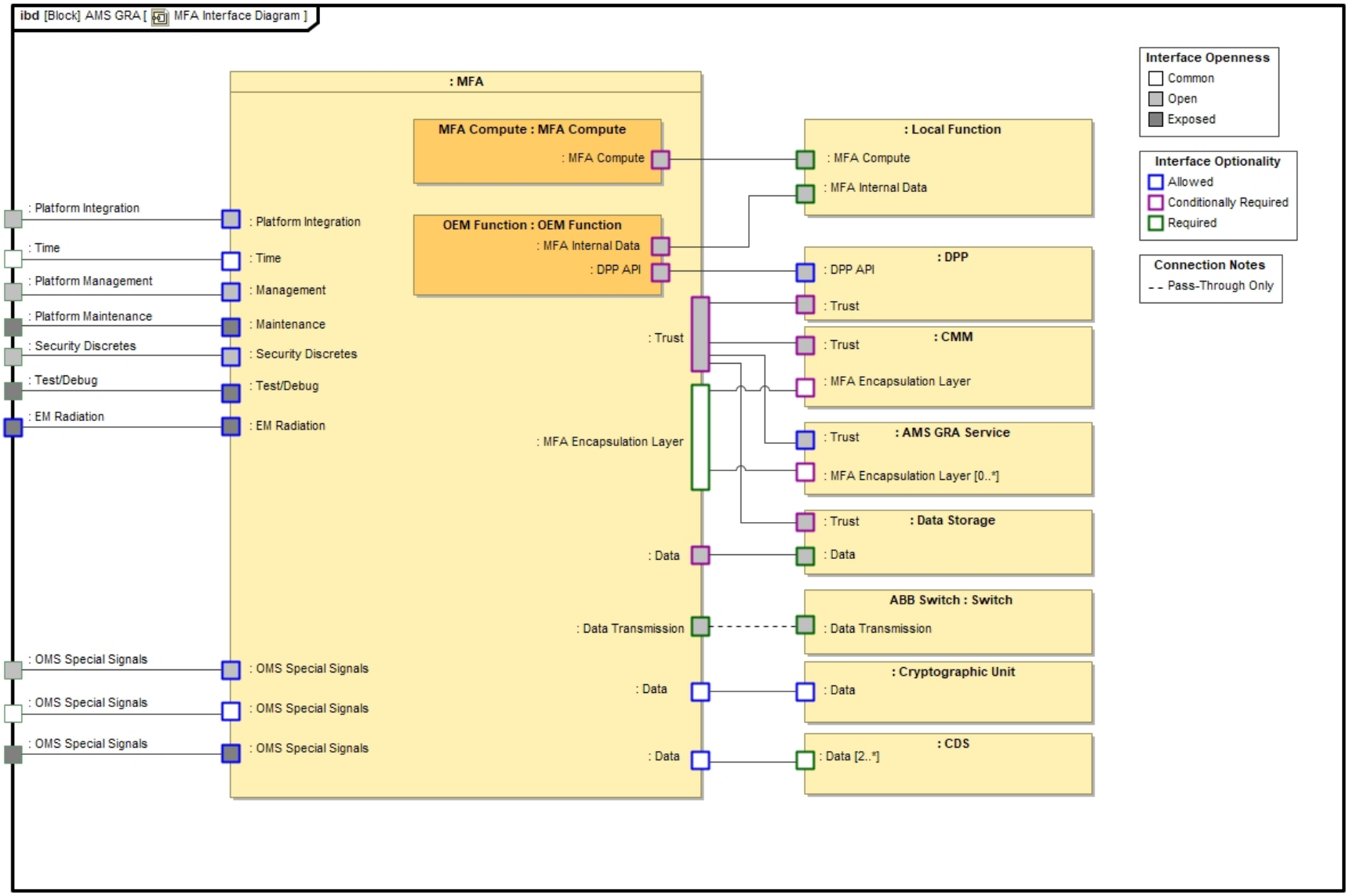


Figure 2.6.1: The MFA MPU Interface Diagram depicts the Interfaces on the MFA MPU and Their Connections.

2.7 Interfaces

Table 2.7.1 identifies the MFA interfaces depicted in **Figure 2.6.1**. When determining whether a given MPU or infrastructure is AMS GRA compliant, Interface Optionality is defined as follows:

- Required interfaces must be implemented

- Conditionally-Required interfaces must be implemented when specific conditions are met.

- Allowed interfaces are optional to implement.

Additional detail is available in the MFA Documentation Template. For interfaces listed multiple times, each instance represents a different openness category an interface could belong to. As an example, the Open Mission Systems (OMS) Special Signals Interface can fall into any of the three openness categories because some MPUs may instantiate an allowable interface at one or more openness levels. Those interfaces will be represented on the corresponding diagram and table for each allowable openness level.

Open Additions to Common Interfaces are permissible without adverse impact to compliance unless explicitly noted. Open Additions to Common Interfaces are allowed to support the opening up of innovative MFA functionality that may be unique to specific implementations. An acceptable Open Addition neither pollutes the Common Interfaces with information that is unique to a specific MFA implementation nor forces the MFA vendor to negotiate unique interface needs with a standards body. Some Open Additions may transition to Common Interfaces over time.

Interface	Documentation	Interface Optionality	Condition
Data	The Data interface specifies data interfaces between the MFA and other MPUs.	Required	N/A
Crypographic Unit (CU) API	The CU API enables AMS GRA Service to access the CU's cryptographic functionalities.	Allowed	N/A
Data Transmission	Data Transmission represents the interface support used to facilitate a data transmission connection. This interface is not concerned with what type of data is being transmitted, it only denotes what MPUs can be connected for transmitting data.	Required	N/A

Interface	Documentation	Interface Optionality	Condition
DPP API	The DPP API provides Level 1 MPUs with access to various facilities provided by the DPP Compute Environment. Note that Level 1 MPUs and MFA OEM functions are architectural elements that deploy to the DPP and make use of this interface, but do not introduce additional requirements beyond how the DPP API is defined.	Conditionally Required	Required for Pattern 3 MFAs, (i.e. an MFA that instantiates its MFA Compute in a DPP)
EM Radiation	An EM Radiation interface is used by the aperture of an MFA for information transfer between EM radiation and electrical signals.	Allowed	N/A
IR MEL API Implementation	This port represents the data exchange between the IR MEL API Implementation and MFA OEM.	Conditionally Required	Required for Pattern 3 MFAs, (i.e. an MFA that instantiates its MFA Compute in a DPP)
Maintenance	The Maintenance interface provides the capability to perform preventative and corrective maintenance while installed on the platform. Example maintenance use cases for the MFA include software/firmware updates, logging, and availability of test/debug.	Allowed	N/A
Management	The Management interface provides the capability to support management of the MPU in an AMS GRA compliant mission system. Example management for the MFA includes Gross Power Control and Bandwidth management.	Allowed	N/A
MFA Compute	The MFA Compute interface allows OEM Functions and Local Functions to interact with the processing resources of the MFA Compute. When the MFA Compute is instantiated within the physical MFA, the MFA Compute interface is an Open interface defined by the MFA provider. When the MFA Compute is instantiated within a DPP utilized by the MFA, the Compute Interface is the DPP API.	Conditionally Required	Required for an MFA that instantiates MFA Compute.

Interface	Documentation	Interface Optionality	Condition
MFA Encapsulation Layer	The MEL defines the common and open set of interfaces and patterns of interaction associated with any AMS GRA compliant MFA. Data exchanges between the MFA and Service need to happen via the MEL over the ABB. (Note: RF MEL and IR MEL details can be found in their respective IDDs.)	Required	N/A
MFA Internal Data	MFA Internal Data is the set of data interfaces provided by the MFA OEM for Local Functions to utilize. These data interfaces are defined by the Local Function DK delivered with the MFA.	Condition-ally Required	Required for an MFA that supports Local Functions.
OMS Special Signals	The Common OMS Special Signals Interface specifies the AMS GRA-tailored Common special signals allowed or required as defined by OMS. These signals include items such as safety interlock, blanking pulse, and transmit inhibit.	Allowed	N/A
OMS Special Signals	The Exposed OMS Special Signals Interface specifies the AMS GRA-tailored, platform-defined special signals allowed or required as defined by OMS. These signals include items such as safety interlock, blanking pulse, and transmit inhibit.	Allowed	N/A
OMS Special Signals	The Open OMS Special Signals Interface specifies the AMS GRA-tailored Open special signals allowed or required as defined by OMS. These signals include items such as safety interlock, blanking pulse, and transmit inhibit.	Allowed	N/A
Platform Integration	The Platform Integration interface represents the general physical interfaces that an MPU will typically have with the host platform, including but not limited to mounting, electrical, hydraulic, and cooling connections.	Allowed	N/A
RF MEL API Implementation	This port represents the data exchange between the RF MEL API Implementation and MFA OEM.	Condition-ally Required	Required for Pattern 3 MFAs, (i.e. an MFA that instantiates its MFA Compute in a DPP)
Security Discretes	The Security interface specifies the security interfaces between the MFA and the platform. This includes items such as zeroize.	Allowed	N/A

Interface	Documentation	Interface Optionality	Condition
Test/Debug	The Test/Debug interface specifies the test/debug interfaces between the MFA and the platform.	Allowed	N/A
Time	The Time Distribution interface specifies the interface to Platform Time.	Allowed	N/A
Trust	The Trust interface enables secure connectivity between endpoints using cryptographic means to establish a secure channel. This channel may be used to verify integrity, authenticate endpoints, or to transfer data requiring higher levels of assurance than normal data transfers, such as CPI, or Software. The implementation of this interface may vary depending on assurance level, exportability, and the various mission specific requirements of the adopting program.	Condition-ally Required	Required if the MPU is used to transfer mission data, software, or associated Key Material, and there are no other interfaces implemented which support integrity verification.

Table 2.7.1: The Procuring Authority, MPU Provider, and MSI Use this Table to Identify the Interfaces for a Compliant MPU or Infrastructure.

2.7.1 MFA Encapsulation Layer (MEL)

The MEL is included in AMS GRA to define the Common set of interfaces and patterns of interaction associated with any AMS GRA-compliant MFA. This approach allows design and integration of innovative and varied MFA implementations into an AMS GRA instantiation. The MEL approach is a critical element to meet many of the business and technical goals of AMS GRA. First, the MEL definition minimizes the need to tailor Skills to reflect specific MFA hardware designs and installation details of one or more MFA faces. This type of MEL definition promotes the independent development and procurement of Skills and MFAs while enabling rapid integration. Second, the MEL approach supports Skill portability between different MFA Types and vendors. Finally, the abstracted interface allows Skills and MFA hardware implementations to evolve independently with minimal ripple effects resulting in rework and retest.

Feature	Benefit
Defines abstracted, Common interfaces and normative functional allocations	Enables independent procurement and upgrading of Skills and MFAs through low coupling of Skills and MFA design.
Encapsulates the details of MFA hardware design choices	Supports innovative MFA designs without imposing additional requirements on other MFAs.
Provides an abstracted, extensible taxonomy for MFA requests	Insulates MFA implementations from the ripple effects of changes required for upgraded or evolving MFAs and promotes rapid capability insertion.
Provides normative data product definitions	Allows Skills to be developed without detailed knowledge of MFA design, enabling a broader pool of Skill developers.
Standardizes interfaces and methods for Skills to interact with MFA	Promotes Skill portability and reuse.

Table 2.7.2: This Table Summarizes the Features and Benefits of the MEL Architectural Element.

The MEL defines the interface between AMS GRA Services and the MFA. The MEL interface is comprised of two sub-elements: The interface definition files provided by the API, and format definitions for data product transfers.

Interface definition files can be included by C++ Skill software. These files contain:

- Command and Control (C2) structures and function calls that are Common to all MFAs within the IR or RF spectrum
- A single abstraction for all MFA implementations which indicates supported features, inclusion of multiple MFA faces and relevant capabilities
- Provisions for control and status between AMS GRA Services and Local Functions if hosted by the MFA
- The equivalent content of a collection of OMS/Universal Command and Control Interface (UCI) messages in sufficient detail that an AMS GRA Service can use the MEL API to bridge required UCI messages to and from the MFA

The MEL provides two format definitions for data product transfers:

- Common formats for allowed MFA data types
- Extendable formats for Local Function data products

The points in the MEL where AMS GRA defines Common content are chosen to balance the objective of AMS

GRA Service portability against the change management risk of demanding too much commonality. Inside of the Common C2 definitions for MFA, AMS GRA allows for necessary customization for MFA design dependent details, such as beamformer weights. In addition to promoting portability, Common data types and formats increase interoperability between MFAs and other MPUs such as a Countermeasures Manager (CMM).

As shown in **Figure 2.6.1**, the MEL provides a Common interface to the MFA for use by AMS GRA Services. The MEL logical interface in **Figure 2.6.1** includes all of the sub-element content defined above. **Figure 2.6.1** also shows that Local Functions can utilize a specific set of MFA OEM Interfaces that provide a Local Function with access to data, as defined by the Local Function DK, that is available for Local Function to process or utilize. These interfaces enable the exchange of data between Local Functions and MFA OEM. The data exchange is non-proprietary; however, the implementation may be proprietary to preserve the competitive trade space of the interfaces internal to an MFA implementation. For example, the implemented interface may use Aurora as the protocol, which is proprietary, to avoid additional translations from one Xilinx card to another. As with all MPUs, behaviors within the MFA may be proprietary (e.g., moving data within the MFA, but not over the MEL or MFA Internal Data interfaces shown in **Figure 2.6.1**).

The MFA provider makes the interfaces and interactions available to a third-party developer through the DK. The DK identifies the locations within the MFA where Local Functions can be hosted and defines any constraints from a processing, memory, or other resource utilization that exist for the specific MFA. Using these interfaces, third parties have an avenue to take full advantage of the flexibility of software-defined apertures. Depending on the specific MFA design, these interfaces can enable concepts such as allowing third-party developers to inject functionality that requires low-latency responses, tailor the digital beamformers to meet new Skill requirements, and format data products to meet specific Skill Open Interfaces. The MEL implementation includes interfaces to enable a service to send and receive data with a Local Function. The MEL implementation is a logical part of the MFA and is delivered as a library for Service development.

2.7.1.1 RF MEL API Partitioning

A partitioned RF MEL API has been instated to align RF MFA functional domains to typical software Service roles. These functional domains are:

- C2: C2 API
- COSITE: RF blanking API
- Data: Data Transfer API
- Direct Endpoint Access (DEA): DEA API
- Monitor: Status Reporting API

Services load API partitions through the use of the software "factory" in the RF MEL API itself. This factory gives MSIs control over the one-to-many RF MEL libraries and API partitions that can be loaded by a particular Service. Reference the RF MEL IDD for more information.

2.7.1.2 MFA Data Types

As part of the MEL definition, both IR and RF MFAs are assigned a set of fundamental data types that describe the expected data products exchanged between MFAs and AMS GRA Services. These types allow the MFA, including the MFA OEM, to provide capabilities described as OEM Functions. Each of these fundamental types has a Common Data format detailed in the respective MEL IDD. In addition to bounding the expected products resulting from MFA OEM Functions, AMS GRA encourages Local Function developers to leverage these defined types and formats to provide data.

Table 2.7.3 describes each of the fundamental data types the IR MFA OEM or IR MFA Local Functions may produce. Further detail on the format definitions is available in the IR MEL IDD.

Name	Documentation
IR Frame	Matrix of pixel data describing a single still image
IRST Track	Continuous location identification information from an IRST MFA
LOS 2D	2D sensor pointing information
LOS 3D	Data result of fusing LOS 2D and Range data
3D Track	Continuous location identification information
Bad Pixel List	A set of pixel locations
Candidate Objects	Objects from multiple detects that pass requirements for declaration
Pre-Candidate Objects	Object data that can be aggregated and processed to form candidate objects

Table 2.7.3: The IR MFA Fundamental Data Types Table Lists and Defines the Specific Functionality Supported by an MFA Instantiation.

Table 2.7.4 describes each of the fundamental data types for RF MFAs. Additional details can be found in the RF MEL IDD.

Name	Documentation
I/Q	Rx or Tx data that may have been pre-processed by channelizers, downsamplers, beamformers, etc.

Name	Documentation
PDW	Data describing pulse detection characteristics such as incident time

Table 2.7.4: The RF MFA Fundamental Data Types Table Lists and Defines the Specific Functionality Supported by an MFA Instantiation.

2.8 Resource Management Utilization of RF Reports

The RF Activity interface, included in the Common RF MEL, communicates the details of RF spectrum usage associated with a specific MFA. Services request notification from the MFA of both scheduled and executing activities. The information provided in this interface can be tailored to specific regions of the RF spectrum and to specific MFA faces. This capability supports both Sensing Skills as well as platform Resource Management functionality. Individual Sensing Skills can use this information to adapt job Requests to avoid higher priority activities and to inform processing functions of ownship emissions and interference. Platform Resource Management functions can use this information to direct or influence tasking in a manner that minimizes conflicts between lower level requests.

3.0 Compliance

3.1 MFA Requirements

Table 3.1.1 gives the applicable MPU compliance requirements for the MFA MPU. All MFA Types are assessed for compliance according to the same requirements. There is no compliance-tiering bias for any MFA Type, as each Type may be used to meet the integration and performance needs of the Mission System in specific ways.

See the Compliance Assessment Approach Volume for more information on the tiered requirement strategy as well as the definitions of the interface characteristics.

MPU developers need to coordinate with the Procuring Authority and applicable MSIs to receive additional guidance beyond basic compliance. This additional guidance might include architectural guidance to ensure the MPU integrates within the overall Mission System as well as guidance to align with Commercial-Off-The-Shelf (COTS) standards such as physical form factor, cooling strategy, and environmental operating conditions.

See **Table 3.2.1** for the relevant AMS GRA DK requirements as referenced below.

See the Mission System Volume for additional details about MFA MPU Fault Reporting.

Note: The verification methods listed in the requirements table are AMS GRA recommendations. An adopting program may alter the recommended verification methods IAW the needs of the program. Refer to the Compliance Assessment Approach Volume for more information.

Id	Text	Compli-ance Tier	Recommended Verification Method	Notes
GRA-MPU-001	An MPU shall be documented IAW the corresponding AMS GRA compliance documentation template or contract template, as applicable.	Tier 1	Inspection	
GRA-MPU-005	An MPU with CPI on Trusted Ops platform shall cryptographically protect CPI using an open implementation.	Tier 3	Analysis	
GRA-MPU-006	An MPU that processes or decrypts CPI on a built-for-export platform shall implement the OMS Standard.	Tier 2	Analysis	
GRA-MPU-007	An MPU that implements the Time interface shall use one or more of PTP, DGT, and OMS legacy timing.	Tier 1	Inspection	
GRA-MPU-008	An MPU shall fully implement the required interface elements defined by the Common MEL API corresponding to its MFA type, but may include Open additions.	Tier 1	Inspection	
GRA-MPU-009	An MPU shall provide one or more software libraries implementing the Common MEL API corresponding to its MFA Type.	Tier 1	Inspection	
GRA-MPU-011	An MPU that hosts Level 1 MPUs shall provide a DK IAW DK requirements.	Tier 1	Inspection	
GRA-MPU-012	An MPU that hosts MFA OEM Functions and Local Functions shall use either MFA Compute hardware outside the CCF, DPP MPUs inside the CCF, or a combination of both.	Tier 3	Inspection	
GRA-MPU-013	An MPU's MEL implementation shall use the ABB for all information exchanges.	Tier 1	Demonstration	
GRA-MPU-022	An MPU shall use the MEL for all information exchanges between AMS GRA Services and MFAs.	Tier 1	Inspection	
GRA-MPU-024	An MPU that interacts with the CDS or Cryptographic Unit shall use the CDS IDD or CU IDD respectively interface for all exchanges with the Cryptographic Unit or CDS.	Tier 1	Inspection	
GRA-MPU-025	An MPU shall report faults with the associated fault status.	Tier 1	Inspection	Off-nominal behavior detailed in compliance documentation.
GRA-MPU-026	An MPU shall implement Required interfaces specified IAW the AMS GRA Model.	Tier 1	Inspection	
GRA-MPU-027	An MPU shall implement all interfaces with the level of Openness specified IAW the AMS GRA Model.	Tier 1	Analysis	

Id	Text	Compli-ance Tier	Recommended Verification Method	Notes
GRA-MPU-029	An MPU shall implement only the interfaces in the corresponding MPU interface table.	Tier 2	Inspection	
GRA-MPU-030	An MPU shall document interface openness IAW the Compliance Assessment Approach.	Tier 1	Inspection	
GRA-MPU-031	An MPU shall include a model compatible with MBSE content contained in the AMS GRA SK release with which the MPU is compliant.	Tier 2	Inspection	
GRA-MPU-032	An MPU shall include a model that identifies all external MPU interfaces.	Tier 2	Inspection	
GRA-MPU-034	An MPU that supports third-party development of VAs shall provide a VA Development Tool.	Tier 1	Inspection	
GRA-MPU-035	An MPU that supports Aperture Virtualization and third-party development of Local Function using the Local Function DK shall provide a VA Development Tool capable of associating the Local Function with a VA.	Tier 1	Inspection	
GRA-MPU-036	An MPU that is capable of hosting Local Functions shall retrieve stored Local Functions from storage designated by the Mission System.	Tier 2	Inspection	
GRA-MPU-039	An MPU shall provide a CLASSIFICATION.md file defining security classification of included file contents.	Tier 2	Inspection	
GRA-MPU-040	An MPU shall provide a DATARIGHTS.md file defining data rights of included file contents.	Tier 2	Inspection	
GRA-MPU-041	An MPU shall document SAST results for all microprocessor software source code in the MPU, IAW the MPU documentation template.	Tier 1	Inspection	
GRA-MPU-042	An MPU shall document DAST results for all microprocessor software in the MPU, IAW the MPU documentation template.	Tier 1	Inspection	
GRA-MPU-043	An MPU hosting Local Functions shall provide the capability in the MFA Compute Interface to obtain current MFA Time.	Tier 1	Inspection	
GRA-MPU-044	An MPU hosting Local Functions shall provide MFA Time for digitized sample data in its MFA Internal Data Interface.	Tier 1	Inspection	
GRA-MPU-045	An MPU hosting Local Functions shall provide the capability to convert an MFA Time to a universal time reference.	Tier 1	Inspection	

Id	Text	Compli-ance Tier	Recommended Verification Method	Notes
GRA-MPU-046	An MPU shall provide automated tests of non-COTS microprocessor software with at least 50% source code structural coverage (excluding third-party COTS or open source libraries).	Tier 1	Demonstration	Only applicable to MPUs that include non-COTS microprocessor code.
GRA-MPU-049	An MPU that implements a scheduling function shall manage the timeline utilization of the MPU.	Tier 1	Demonstration	
GRA-MPU-050	An MPU with a scheduling function, that instantiates VAs, shall manage the timeline utilization of the VAs.	Tier 1	Demonstration	
GRA-MPU-058	An MPU shall encrypt classified DaR using NSA-approved algorithms.	Tier 1	Analysis	
GRA-MPU-059	An MPU that decrypts and/or encrypts classified data and/or CPI shall only store classified plain text keys in volatile memory.	Tier 1	Inspection	
GRA-MPU-060	An MPU's write-protected memory shall be modifiable through a hardware element in addition to software.	Tier 1	Analysis	
GRA-MPU-061	An MPU shall ensure the authenticity and integrity of MPU software and firmware during MPU initialization.	Tier 1	Inspection	
GRA-MPU-062	An MPU shall support integrity verification to the Mission System over a secure communication channel.	Tier 1	Demonstration	
GRA-MPU-063	An MPU shall implement secure communications using open standards.	Tier 1	Analysis	
GRA-MPU-064	An MPU shall use NSA-approved algorithms to authenticate and verify the integrity of a software load when received over a non-secure communication channel.	Tier 1	Demonstration	
GRA-MPU-065	An MPU shall use NSA-approved algorithms to authenticate and verify the integrity of software locally stored at rest, not including bootloaders; prior to updating, loading, and executing.	Tier 1	Inspection	
GRA-MPU-066	An MPU shall support the use of a Mission System-provided RoT.	Tier 1	Demonstration	
GRA-MPU-067	An MPU shall publish details of security relevant events in an open format.	Tier 1	Demonstration	
GRA-MPU-068	An MPU shall boot from write-protected memory.	Tier 1	Demonstration	

Id	Text	Compli-ance Tier	Recommended Verification Method	Notes
GRA-MPU-069	An MPU shall format MEL data IAW the MEL IDD corresponding to the MFA Type. Open additions are permitted.	Tier 1	Demonstration	
GRA-MPU-074	An MPU that processes or decrypts CPI on a built-for-export platform shall fully implement OMS Standard.	Tier 3	Analysis	
GRA-MPU-075	An MPU shall implement the required interface elements defined by the Common MEL API corresponding to its MFA Type, and does not include Open additions.	Tier 2	Inspection	
GRA-MPU-077	An MPU shall provide automated tests of non-COTS microprocessor software with at least 80% source code structural coverage (excluding third-party COTS or open source libraries).	Tier 3	Demonstration	
GRA-MPU-078	An MPU model shall be capable of automated verification with AMS GRA models.	Tier 3	Inspection	
GRA-MPU-079	An MPU shall include a model that provides definition of all external MPU interfaces.	Tier 3	Inspection	
GRA-MPU-080	An MPU shall format MEL data IAW the MEL IDD corresponding to the MFA Type.	Tier 2	Demonstration	
GRA-MPU-085	An MPU shall report faults with the associated fault status using the applicable interface. (See notes).	Tier 2	Inspection	Cryptographic Unit MPUs and CDS MPUs report faults as defined in the CDS and CU IDD. Local Function MPUs report faults as defined in Local Function DK. MFA MPUs report faults as defined in the applicable MEL IDD. AMS GRA Service, CMM, DPP, Data Storage, Switch, OMS Service, and PCaB MPUs report faults using a software interface in UCI format.

GRA-MPU-086	An MPU shall use the CMM IDD for all information exchanges between the MPU and the CMM.	Tier 1	Inspection	
GRA-MPU-092	An MPU shall include a software bill of materials in an open and non-proprietary format.	Tier 2	Inspection	

Table 3.1.1: An MFA MPU's Adherence to the Compliance Requirements Determines Its Compliance Score. See the Compliance Assessment Approach Volume for More Details.

3.2 Development Kit (DK) Requirements

Table 3.2.1 provides DK requirements that apply to DPP and MFA MPUs. These requirements are part of the authoritative AMS GRA compliance assessment. The table indicates the processor types (i.e., General Purpose Processor (GPP), Graphics Processing Unit (GPU), and Field-Programmable Gate Array (FPGA)) to which each requirement applies in order to assist the respective DK definition activity. In some cases, the MSI may replace the content in a requirement originally provided by the DPP or MFA MPU provider. The column for MSI indicates which requirements the MSI may serve as an alternate provider of the content.

There are three patterns that refine some of the DK requirement definitions in **Table 3.2.1**. First, some requirements exist in pairs such that one requirement discusses a need for a commodity item while the other requirement presents the need for a custom tool. Second, and related to the first, some requirements only request the identification of certain DK content while others require the actual provision of the content. For example, a DK provider that uses a commodity item with a license would identify the item, so that a DK user could procure the item and acquire a license. A DK provider that uses a custom item would need to provide the item to the DK user. Third, some requirements include the phrase “if required for development.” These requirements may or may not apply to the compliance assessment–depending on the DK provider’s implementation of the DK. If the DK implementation enables development without the need for a requirement that is marked in this way, then the requirement does not apply.

For the tier definitions of some DK requirements, AMS GRA applies a versioning philosophy. Tier 3 compliance requires the identification of a specific version of a standard. For example, hardware form factor, programming language, and network protocol, or development tool, provided the standard or tool is backward compatible. The identification of specific versions may occur to support testing purposes or to support a lack of backward compatibility of features used by AMS GRA. AMS GRA instantiations should enable version updates that eliminate reliance on deprecated or removed features in obsolete versions of standards or tools.

See the Software Architecture Volume for more information regarding DKs and the overall software development environment.

Note: The verification methods listed in the requirements table are AMS GRA recommendations. An adopting program may alter the recommended verification methods IAW the needs of the program. Refer to the Compliance Assessment Approach Volume for more information.

Id	Text	Compli-ance Tier	Recommended Verification Method	GPP	GPU	FPGA	MSI	Notes
GRA-DK-001	A DK shall provide a developer guide and support documentation, including a Software Bill of Materials, for a developer to use the DK and integrate within a DevSecOps tool chain.	Tier 1	Inspection	Yes	Yes	Yes	No	
GRA-DK-002	A DK shall include a build environment with pre-installed build infrastructure (where licensing/redistribution restrictions allow) if required for development.	Tier 1	Inspection	Yes	Yes	Yes	Yes	Applies only to DPP Build Infrastructure could consist of virtual machines and/or containers
GRA-DK-003	A DK shall include a development environment with libraries and tools pre-installed (where licensing/redistribution restrictions allow) if required for development.	Tier 1	Inspection	Yes	Yes	Yes	Yes	Development environment could consist of virtual machines or containers
GRA-DK-004	A DK shall provide simulators if required for development.	Tier 1	Inspection	Yes	Yes	Yes	Yes	
GRA-DK-005	A DK shall provide emulators if required for development.	Tier 1	Inspection	Yes	Yes	Yes	Yes	
GRA-DK-006	A DK shall provide software configuration management support to facilitate integration of software into the DevSecOps environment.	Tier 1	Inspection	Yes	Yes	Yes	Yes	

Id	Text	Compli-ance Tier	Recommended Verification Method	GPP	GPU	FPGA	MSI	Notes
GRA-DK-007	A DK shall provide build infrastructure configuration to configure the build environment.	Tier 1	Inspection	Yes	Yes	No	Yes	Infrastructure configuration could be provided either as documentation or as code Updated to "build infrastructure" and "build environment". The term "build environment" was used to allow developers to define their development workspace.
GRA-DK-008	A DK shall provide a Service Framework Reference Implementation for the design and implementation of a Skill.	Tier 3	Inspection	Yes	Yes	Yes	No	
GRA-DK-009	A DK shall provide a container framework if required for development.	Tier 1	Inspection	Yes	No	No	Yes	
GRA-DK-010	A DK shall provide a unit testing framework for a DPP. This framework enables the creation of unit tests for Level 1 MPUs.	Tier 1	Inspection	Yes	Yes	Yes	No	
GRA-DK-011	A DK shall provide target support packages to configure the specific processing resource.	Tier 1	Inspection	Yes	Yes	Yes	No	
GRA-DK-012	A DK shall identify specific versions of commodity build support utilities in a machine-readable format.	Tier 1	Inspection	Yes	Yes	Yes	Yes	An SBOM is a means to achieve machine readability.
GRA-DK-013	A DK shall provide custom build support utilities if required for development.	Tier 1	Inspection	Yes	Yes	Yes	Yes	
GRA-DK-014	A DK shall provide documentation of compiler flags and/or configuration properties if required for development.	Tier 1	Inspection	Yes	Yes	No	Yes	

Id	Text	Compli-ance Tier	Recommended Verification Method	GPP	GPU	FPGA	MSI	Notes
GRA-DK-015	A DK shall provide test tools for applicable Level 1 MPUs.	Tier 1	Inspection	Yes	Yes	Yes	Yes	The level 2 DPP MPU provides a DK for applicable level 1 MPUs such as AMS or OMS services.
GRA-DK-016	A DK shall identify specific versions of commodity tools if required for development in a machine-readable format.	Tier 1	Inspection	Yes	Yes	Yes	Yes	An SBOM is a means to achieve machine readability.
GRA-DK-017	A DK shall provide custom tools if required for development.	Tier 1	Inspection	Yes	Yes	Yes	Yes	
GRA-DK-018	A DK shall identify software integrity tools for a DPP.	Tier 1	Inspection	Yes	Yes	Yes	Yes	
GRA-DK-019	A DK shall provide custom debug tools if required for development.	Tier 1	Inspection	Yes	Yes	Yes	Yes	
GRA-DK-020	A DK shall identify specific versions of commodity packaging scripts and/or tools in a machine-readable format.	Tier 1	Inspection	Yes	Yes	Yes	Yes	An SBOM is a means to achieve machine readability.
GRA-DK-021	A DK shall provide custom packaging scripts and/or tools if required for development.	Tier 1	Inspection	Yes	Yes	Yes	Yes	
GRA-DK-022	A DK shall identify DASTs if required for development.	Tier 3	Inspection	Yes	Yes	Yes	Yes	
GRA-DK-023	A DK shall identify SASTs and associated configurations using IaC if required for development.	Tier 3	Inspection	Yes	Yes	Yes	Yes	
GRA-DK-024	A DK shall identify specific versions of commodity release packaging in a machine-readable format.	Tier 1	Inspection	Yes	Yes	Yes	Yes	An SBOM is a means to achieve machine readability.
GRA-DK-025	A DK shall provide custom release packaging if required for development.	Tier 1	Inspection	Yes	Yes	Yes	Yes	
GRA-DK-026	A DK shall identify specific versions of commodity dependent documentation and/or libraries in a machine-readable format if required for development.	Tier 1	Inspection	Yes	Yes	Yes	Yes	An SBOM is a means to achieve machine readability.

Id	Text	Compli-ance Tier	Recommended Verification Method	GPP	GPU	FPGA	MSI	Notes
GRA-DK-027	A DK shall provide specialty infrastructure libraries if required for development.	Tier 1	Inspection	Yes	Yes	No	Yes	
GRA-DK-028	A DK shall provide MASI elements whose behavior is required for development.	Tier 1	Inspection	Yes	No	No	Yes	
GRA-DK-029	A DK shall provide the processing resource tool configuration if required for development in a machine-readable format.	Tier 1	Inspection	Yes	Yes	Yes	Yes	
GRA-DK-030	A DK shall provide FPGA intellectual property Blocks if required for development.	Tier 1	Inspection	No	No	Yes	Yes	
GRA-DK-032	When used for DPP development, the DK shall provide a DPP API implementation required for development.	Tier 1	Inspection	Yes	No	No	Yes	Machine readable is supposed to be an SBOM
GRA-DK-033	A DK shall provide CI/CD support configuration files necessary to integrate DK components into a DevSecOps pipeline.	Tier 1	Inspection	Yes	Yes	No	Yes	
GRA-DK-034	A DK shall provide available OEM Function interfaces when used for Local Function development.	Tier 1	Inspection	Yes	Yes	Yes	No	
GRA-DK-035	A DK shall include a containerized build environment with pre-installed build infrastructure (where licensing/redistribution restrictions allow) if required for development.	Tier 2	Inspection	Yes	Yes	Yes	Yes	Applies only to DPP Build Infrastructure must consist of containers only
GRA-DK-036	A DK shall include a containerized build environment with pre-installed build infrastructure (where licensing/redistribution restrictions allow) if required for development.	Tier 3	Inspection	Yes	Yes	Yes	Yes	Applies only to MFA Build Infrastructure must consist of containers only
GRA-DK-037	A DK development environment shall include debugging tools and libraries created by the MPU provider to streamline development for the target MPU.	Tier 3	Inspection	Yes	No	No	No	
GRA-DK-038	A DK shall provide build infrastructure configuration as IaC to configure the build environment.	Tier 3	Inspection	Yes	Yes	No	Yes	

Id	Text	Compli-ance Tier	Recommended Verification Method	GPP	GPU	FPGA	MSI	Notes
GRA-DK-039	A DK shall provide a unit testing framework for an MFA. This framework enables the creation of unit tests for Level 1 MPUs.	Tier 3	Inspection	Yes	Yes	Yes	No	
GRA-DK-040	A DK shall provide documentation (as `Cmake` or equivalent project settings as code) of compiler flags and/or configuration properties if required for development.	Tier 2	Inspection	Yes	Yes	Yes	Yes	
GRA-DK-041	A DK shall document all licensed/non-redistributable tools and libraries required for the development environment.	Tier 1	Inspection	Yes	Yes	Yes	Yes	
GRA-DK-042	A DK shall document all licensed/non-redistributable software required for the build environment.	Tier 1	Inspection	Yes	Yes	Yes	Yes	Applies only to DPP
GRA-DK-043	A DK shall document all licensed/non-redistributable software required for the build environment.	Tier 3	Inspection	Yes	Yes	Yes	Yes	Applies only to MFA
GRA-DK-044	A DK shall identify specific versions of commodity packaging scripts and/or tools.	Tier 3	Inspection	Yes	Yes	Yes	Yes	Applies only to MFA

Table 3.2.1: A DK's Adherence to the Compliance Requirements Determines Its Compliance Score. See the Compliance Assessment Approach Volume for More Details.

A.0 Acronyms

ABB	Avionics Backplane Bus
ADC	Analog to Digital Converter
AMS	Agile Mission Suite
API	Application Programming Interface
BIT	Built-In Test
C2	Command and Control
CCF	Compute Cluster Farm
CDS	Cross Domain Solution
CI/CD	Continuous Integration/Continuous Delivery
CMM	Countermeasures Manager
COTS	Commercial-Off-The-Shelf
CPI	Critical Program Information
CU	Crypographic Unit
DAC	Digital to Analog Converter
DaR	Data at Rest
DAST	Dynamic Application Security Testing
DEA	Direct Endpoint Access
DevSecOps	Development, Security, and Operations
DGT	Distributed Guaranteed Time
DK	Development Kit
DPP	Digital Payload Processor
EM	Electromagnetic
FoR	Field of Regard
FPA	Focal Plane Array
FPGA	Field-Programmable Gate Array
GPP	General Purpose Processor
GPU	Graphics Processing Unit

GRA	Government Reference Architecture
I/O	Input/Output
I/Q	In-phase/Quadrature
IaC	Infrastructure as Code
IAW	In Accordance With
IDD	Interface Description Document
IQ	In-Phase and Quadrature
IR	Infrared
IRST	Infrared Search and Track
LOS	Line of Sight
MASI	Mission Agnostic Service Infrastructure
MBSE	Model-Based Systems Engineering
MEL	MFA Encapsulation Layer
MFA	Multi-Function Aperture
MFP	Multi-Function Processor
MPS	Mission Processing Subsystem
MPU	Minimum Procurable Unit
MSI	Mission System Integrator
NSA	National Security Agency
OEM	Original Equipment Manufacturer
OMS	Open Mission Systems
PCaB	Processor Chassis and Backplane
PDW	Pulse Descriptor Word
PTP	Precision Time Protocol
RF	Radio Frequency
RoCE	RDMA over Converged Ethernet
RoT	Root of Trust
Rx	Receive
SAST	Static Application Security Testing

SBOM	Software Bill of Materials
SK	Starter Kit
SRM	Sensor Resource Management
Tx	Transmit
UCI	Universal Command and Control Interface
VA	Virtual Aperture
VADB	Virtual Aperture Database
VITA	VME bus International Trade Association
VME	Versa Module Eurocard

B.0 Revision History

The Revision History table, **Table B.0.1**, captures the major changes made for a given AMS GRA volume as the Starter Kit (SK) continues to mature.

Revision	Description	Date	Notes
-	This revision adds a revision table to the volume.	May 22, 2024	SK11.0
A	This revision adds the AMS GRA Model as an authoritative reference and provides prose updates and clarifications.	July 24, 2025	SK13.0
B	This revision removes references to the security module. Updates GRA-MPU-008 for clarity. Adds new requirement GRA-MPU-092.	April 24, 2026	AMS GRA 14.0

Table B.0.1: Revision History Captures the Major Changes Made to the MFA MPU Between Documentation Releases.